

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Малышенко А. Р.

Группа К34392

Проверил:

Добряков Д. И.

Санкт-Петербург

2024 г.

Содержание отчета

1) Постановка задачи	3
2) Ход работы	3
3) Вывод	9

1) Постановка задачи

Нужно написать свой boilerplate на express + sequelize / TypeORM + typescript.

Должно быть явное разделение на:

- модели
- контроллеры
- роуты
- сервисы для работы с моделями (реализуем паттерн “репозиторий”)

2) Ход работы

Создаем две модели: модель User (рисунок 1) – пользователь в системе, необходима в том числе для авторизации пользователя и модель Product (рисунок 2) – модель абстрактного продукта.

```
export type UserAttributes = {
  id: number,
  name: string,
  email: string,
  password: string
}

export type UserCreate = Optional<UserAttributes, "id">

@Table Show usages
export class User extends Model<UserAttributes, UserCreate> {
  @AllowNull(false)
  @Column
  name!: string

  @Unique
  @AllowNull(false)
  @Column
  email!: string

  @AllowNull(false)
  @Column
  password!: string

  @BeforeCreate Show usages
  @BeforeUpdate
  static hashPassword(instance: User) {
    const handler = new PasswordHandler()
    const { password } = instance
    if (instance.changed("password")) {
      instance.password = handler.hashPassword(password)
    }
  }
}

export default User Show usages
```

Рисунок 1 – модель User

```

export type ProductAttributes = {
  id: number,
  name: string,
  description: string,
}

export type ProductCreate = Optional<ProductAttributes, "id">

@Table Show usages
export class Product extends Model<ProductAttributes, ProductCreate> {
  @AllowNull(false)
  @Column
  name!: string

  @AllowNull(false)
  @Column
  description!: string
}

export default Product Show usages

```

Рисунок 2 – модель Product

Далее настраиваем подключение к базе данных (рисунок 3), в данном случае используется sqlite.

```

const sequelize: Sequelize = new Sequelize({
  username: "root",
  password: "",
  database: "database_development",
  host: "127.0.0.1",
  dialect: 'sqlite',
  storage: "database.sqlite",
  repositoryMode: true
})

const models = [User, Product]

sequelize.addModels(models)
sequelize.sync()
  .then(() => console.log("models are synced"))
  .catch((error) => console.log(error))

export default sequelize Show usages

```

Рисунок 3 – подключение к бд

Настраиваем роутеры в routers/index.ts (рисунок 4) и роутеры для users и products (рисунок 5).

```
const router: express.Router = express.Router()

router.use('/users', userRoutes)
router.use('/products', productRoutes)

export default router
```

Рисунок 4 – роутер в index.ts

```
import express from "express"
import UserController from "../../controllers/user/userController"
import UserService from "../../services/user/userService";

const controller = new UserController(new UserService());
const userRoutes = express.Router();

userRoutes.route("/").post(controller.post);
userRoutes.route("/").get(controller.getAll);
userRoutes.route("/:user_id").get(controller.get);
userRoutes.route("/:user_id").put(controller.update);
userRoutes.route("/:user_id").delete(controller.delete);

export default userRoutes
```

Рисунок 5 – роутер User.ts

Далее создаем контроллеры (рисунок 6, 7), сервисы (рисунок 8, 9), маперы (рисунок 10), в качестве примера, скриншоты ниже представлены только для модели user.

```

class UserController { Show usages
  private readonly userService: UserService
  private readonly userMapper: UserMapper = new UserMapper()

  constructor(userService: UserService) { Show usages
    this.userService = userService
  }

  getAll = async (request: Request, response: Response) => { Show usages
    const userList = await this.userService.getAll()
    if (!userList) {
      return response.status(404).send()
    }

    let data: any[] = []

    userList.forEach( user => {
      data.push(this.userMapper.userToDict(user))
    })

    return response.status(200).json(data)
  }

  get = async (request: Request, response: Response) => { Show usages
    const id = Number(request.params.user_id)
    const user = await this.userService.findById(id)
    if (!user) {
      return response.status(404).send()
    }

    const data = this.userMapper.userToDict(user)
    return response.status(200).json(data)
  }

  post = async (request: Request, response: Response) => { Show usages
    const body: UserCreate = request.body

    try {
      const createdUser = await this.userService.create(body)
      const data = this.userMapper.userToDict(createdUser)
      return response.status(201).send(data)
    }
  }
}

```

Рисунок 6 – контроллер для модели user

```

    }
  }

  delete = async (request: Request, response: Response) => { Show usages
    const id = Number(request.params.user_id)
    const deletedRows = await this.userService.deleteById(id)
    if (deletedRows === 0) {
      return response.status(404).send()
    }

    return response.status(204).send()
  }

  update = async (request: Request, response: Response) => { Show usages
    const body: UserCreate = request.body
    const id = Number(request.params.user_id)
    try {
      await this.userService.updateById(id, body)
      const user = await this.userService.findById(id)
      if (!user) {
        return response.status(404).send()
      }

      const data = this.userMapper.userToDict(user)
      return response.status(200).send(data)
    } catch (e: any) {
      return response.status(400).send({message: e.message})
    }
  }
}

export default UserController Show usages

```

Рисунок 7 – продолжение рисунка 6

```

class UserService implements CrudService<number, User, UserCreate> { Show usages
  private userRepository: Repository<User> = sequelize.getRepository(User)

  create(data: UserCreate): Promise<User> { Show usages
    try {
      return this.userRepository.create(data)
    } catch (e: any) {
      return Promise.reject(e)
    }
  }

  deleteById(id: number): Promise<number> { Show usages
    return this.userRepository.destroy({
      where: {
        id: id
      }
    })
  }
}

```

Рисунок 8 – сервис для модели user

```

    async findById(id: number): Promise<User | null> { Show usages
        return this.userRepository.findOne({
            where: {
                id: id
            }
        })
    }

    async getAll(): Promise<Array<User> | null> { Show usages
        return this.userRepository.findAll()
    }

    async updateById(id: number, data: UserCreate): Promise<User> { Show usages
        const result = await this.userRepository.update(data, {
            where: {
                id: id
            },
            returning: true
        })

        if (result[0] === 0) {
            return Promise.reject({message: "Not found"})
        }

        return Promise.resolve(result[1][0])
    }
}

export default UserService Show usages

```

Рисунок 9 – продолжение рисунка 8

```

class UserMapper { Show usages
    userToDict(user: User) { Show usages
        return {
            id: user.id,
            name: user.name,
            email: user.email
        };
    }
}

export default UserMapper; Show usages

```

Рисунок 10 – мапер для модели user

3) Вывод

В ходе лабораторной работы №1 был реализован boilerplate на express с использованием языка typescript. Для этого были реализованы 2 модели и для них, контроллеры, сервисы, маперы и роутеры.